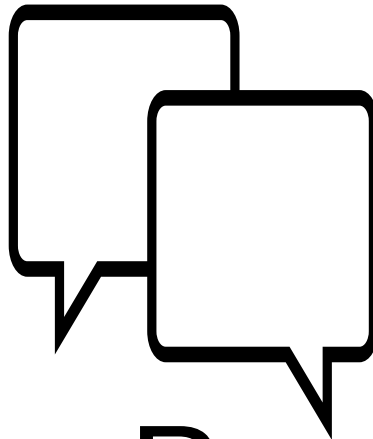




Inter Process Communication

Gestire l'interazione tra processi

Lezione 24
2022/23
Corso 1



Relazione tra processi

- In certe situazioni è sufficiente interagire in modi semplici:
 - Attendere che un processo termini e leggere l'exit code
 - Comunicare tramite stdin e stdout
- In generale è possibile che 2 processi (non necessariamente legati da parentela padre/figlio) vogliano scambiarsi informazioni e/o vogliano sincronizzarsi.

IPC - InterProcess Communication

- Il S.O. impedisce il trasferimento diretto di dati tra processi
 - Ogni processo dispone di uno spazio di indirizzamento separato
 - Non è possibile sapere cosa sta capitando in un altro processo
- Ogni S.O. offre alcuni meccanismi per superare tale barriera in modo controllato
 - Permettendo lo scambio di dati...
 - ...e la sincronizzazione delle attività

Rappresentazione delle informazioni scambiate

- I due processi che comunicano sono soggetti indipendenti
- Occorre adattare le informazioni scambiate, così da renderle comprensibili al destinatario

Rappresentazione interna

- La rappresentazione interna è un fatto *privato* (ognuno come meglio crede)
- Internamente, un processo può usare una varietà di rappresentazioni
 - Tipi elementari (numerici, logici, caratteri, ...)
 - Tipi strutturati (record, array, classi, ...)
 - Puntatori per strutture dati complesse (alberi, grafi, ...)
- La rappresentazione interna non è adatta ad essere esportata
 - Anche i tipi di dati elementari possono avere rappresentazioni differenti (ad es. le stringhe)
 - I puntatori non hanno senso al di fuori del proprio spazio di indirizzamento (ad es. una linked list non può essere esportata)
 - Alcune informazioni (handle) non sono esportabili perché hanno senso solo nel contesto in cui è stata creata
- Dobbiamo tradurre la struttura della rappresentazione interna in modo che sia trasferibile e interpretabile.

Rappresentazione esterna

- Formato intermedio che permette la rappresentazione di strutture dati arbitrarie
 - Sostituendo i puntatori con riferimenti indipendenti dalla memoria
- Formati basati su testo
 - XML, JSON, CSV, ...
- Formati binari (più compatti, ma più difficili da interpretare e debuggare)
 - XDR, HDF, protobuf ...

Formato utilizzato dalle librerie gRPC (Google Remote Procedure Calls).

gRPC framework open-source che permette di creare formati Protobuf da/verso qualunque linguaggio. Fornisce un ambiente di programmazione remota molto efficace.

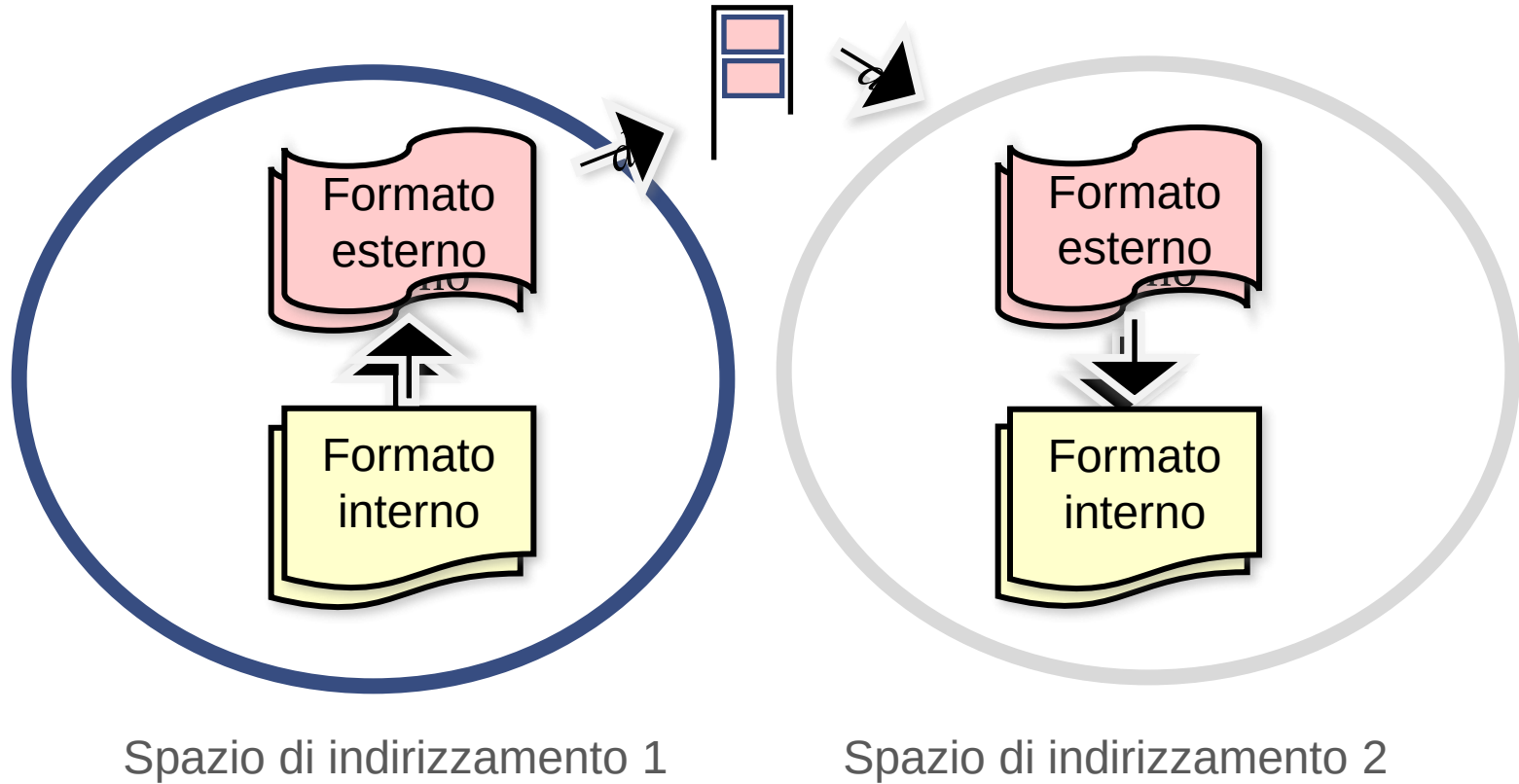
Serializzazione

- Le rappresentazioni esterne possono essere trattate come blocchi compatti di byte
 - Possono essere duplicati e trasferiti senza comprometterne il significato
- I dati vengono scambiati nel formato esterno
 - La sorgente esporta le proprie informazioni (marshalling)
 - Il destinatario ricostruisce una rappresentazione su cui può operare direttamente (unmarshalling)
- La ricostruzione operata dal destinatario può corrispondere o meno a quella utilizzata dalla sorgente
 - In certi casi può essere conveniente trasformare la struttura dati in un formato più consono alle operazioni che si intendono fare su di essa
- Le operazioni di marshalling e unmarshalling possono essere codificate esplicitamente dal programmatore
 - O essere eseguite da codice generato automaticamente dall'ambiente di sviluppo

Coda di messaggi

- Struttura dati mantenuta dal sistema operativo che permette a molti processi sorgente di inviare messaggi ad uno specifico destinatario
 - La comunicazione è di tipo asincrono, il sistema operativo si fa carico di memorizzare il messaggio inviato fino a che il destinatario non lo leggerà
 - La comunicazione è mono-direzionale
- Ogni coda è caratterizzata da *un identificativo univoco* a livello di sistema operativo (una stringa)
 - Il destinatario della coda è responsabile di inizializzarla e legarla al suo nome
 - Le sorgenti devono avere modo di conoscere a priori il nome della coda o servirsi di altri meccanismi di IPC per venirne a conoscenza
- I messaggi sono trasmessi e letti a blocchi
- Gli oggetti "mailslot" in Windows e "fifo" in Linux ricadono in questa categoria
 - Vengono costruiti in C/C++ tramite le funzioni `CreateMailslot(...)` e `mkfifo(...)`

Coda di messaggi



Named pipe (Linux)

- **Named pipe**, chiamate anche FIFO (first in, first out) offrono i seguenti vantaggi:
 - esistono nel file system
 - possono essere utilizzate da processi *che non hanno relazione di parentela padre/figlio*
 - esistono fino a quando non sono esplicitamente cancellati
 - hanno una capacità di buffer elevate (tipicamente intorno a 40Kbyte)
- Sono utilizzabili solo in modo unidirezionale
- Una named pipe è creata con il comando `mkfifo`.
- La named pipe deve essere inizialmente aperta con la funzione `open()`
- Le funzioni di scrittura aggiungono dati all'inizio della coda FIFO
- Le funzioni di lettura prelevano dati dalla fine della coda FIFO.

```

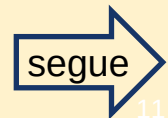
use std::io;
use std::process::Command;
use std::fs::OpenOptions;
use std::io::Write;
use std::path::Path;
fn main() {
    let pipe_path = "/tmp/my_pipe";
    Command::new("mkfifo") // Avvia il processo lettore
        .arg(&pipe_path)
        .spawn()
        .expect("Failed to start child process");

    Command::new("reader") // Avvia il processo lettore
        .env("PATH", "./target/debug/")
        .spawn()
        .expect("Failed to start child process");

    // Check if the named pipe exists
    if ! Path::new(pipe_path).exists() {
        eprintln!("Named pipe {} does not exist. Please create it first using mkfifo.", pipe_path);
        return;
    }

    // Open the named pipe for writing
    let mut pipe = match OpenOptions::new().write(true).open(pipe_path) {
        Ok(pipe) => pipe,
        Err(e) => {
            eprintln!("Failed to open named pipe: {}", e);
            return;
        }
    };
};

```



11

```

loop {
    let mut input = String::new();
    match io::stdin().read_line(&mut input) {
        Ok(0) => break, // Ctrl+D
        Ok(_) => {
            if let Err(e) = pipe.write_all(input.as_bytes()) {
                eprintln!("Errore nell'invio: {}", e);
                break;
            }
            let _ = pipe.flush();
        }
        Err(e) => {
            eprintln!("Errore nella lettura input: {}", e);
            break;
        }
    }
}

```



```
use std::fs;
use std::fs::OpenOptions;
use std::io::{BufRead, BufReader};
```

```
fn main() {
    let pipe_path = "/tmp/my_pipe";

    // Open the named pipe for reading
    let pipe = match OpenOptions::new().read(true).open(pipe_path) {
        Ok(pipe) => pipe,
        Err(e) => {
            eprintln!("Failed to open named pipe: {}", e);
            return;
        }
    };

    // Create a buffered reader for the pipe
    let reader = BufReader::new(pipe);

    // Read from the pipe line by line
    for line in reader.lines() {
        match line {
            Ok(content) => println!("Received: {}", content),
            Err(e) => eprintln!("Failed to read from named pipe: {}", e),
        }
    }
    let _ = fs::remove_file(pipe_path) ;
}
```



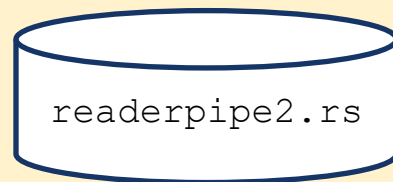
Comunicazione semisincrona

- Una named pipe ha una dimensione finita
- Se la named pipe è piena il mittente è bloccato nella scrittura

```
use std::fs::OpenOptions;
use std::io::Read;
use std::time::Duration;

fn main() {
    let path = "/tmp/mia_fifo";
    let mut file = OpenOptions::new().read(true).open(path).expect("Impossibile aprire la FIFO");

    let mut buffer = [0u8; 4096];
    std::thread::sleep(Duration::from_secs(10));
    println!("Lettore avviato. Inizio lettura...");
    loop {
        match file.read(&mut buffer) {
            Ok(0) => {
                println!("EOF raggiunto");
                break;
            }
            Ok(n) => println!("Letti {} byte", n),
            Err(e) => {
                eprintln!("Errore di lettura: {}", e);
                break;
            }
        }
    }
}
```



```

use std::fs::OpenOptions;
use std::io::Write;
fn main() {
    let path = "/tmp/mia_fifo";
    std::process::Command::new("mkfifo").arg("/tmp/mia_fifo").status().unwrap();
    std::process::Command::new("reader") // Avvia il processo lettore
        .env("PATH", "./target/debug/")
        .spawn()
        .expect("Failed to start child process");
    let mut file = OpenOptions::new().write(true).open(path).expect("Impossibile aprire la FIFO");
    let block = vec![b'a'; 8192]; // blocco da 8KB
    let mut count = 0;
    println!("Inizio scrittura nella FIFO...");
    loop {
        match file.write_all(&block) {
            Ok(_) => {
                count += 1;
                println!("Scritti {} blocchi ({} KB)", count, count * 8);
            }
            Err(e) => {
                eprintln!("Errore di scrittura: {}", e);
                break;
            }
        }
        if count == 100 {
            break;
        }
    }
}

```



writerpipe2.rs

Socket (Linux)

- **Socket** locale
- Bidirezionale
- Può avere un path nel file system
- Supporta connessioni multiple (come un server TCP accetta più client)
- Adatto per una gestione client/server


```

use std::os::unix::net::{UnixListener, UnixStream};
use std::path::Path;
use std::io::{Read, Write};

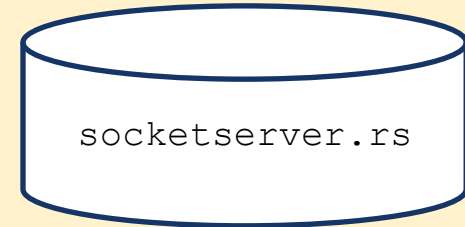
fn handle_server(mut stream: UnixStream) {
    let mut buffer = [0u8; 5];
    match stream.read(&mut buffer) {
        Ok(size) => {
            println!("Messaggio ricevuto: {}", String::from_utf8_lossy(&buffer[..size]));
            stream.write_all(&buffer[..size]).unwrap();
        }
        Err(e) => eprintln!("Errore nella lettura: {}", e),
    }
}

fn main() -> std::io::Result<()> {
    let socket_path = "/tmp/echo.sock";
    if Path::new(socket_path).exists() {
        std::fs::remove_file(socket_path)?;
    }

    let listener = UnixListener::bind(socket_path)?;
    println!("Server in ascolto su {}", socket_path);

    for stream in listener.incoming() { // in server rimane in attesa di una connessione
        match stream {
            Ok(stream) => handle_server(stream),
            Err(err) => eprintln!("Errore nella connessione: {}", err),
        }
    }
    Ok(())
}

```



```
use std::os::unix::net::UnixStream;
use std::io::{Read, Write};

fn main() -> std::io::Result<()> {
    let mut stream = UnixStream::connect("/tmp/echo.sock")?;
    let messaggio = b"Ciao dal client!";
    stream.write_all(messaggio)?;

    let mut buffer = [0u8; 512];
    let size = stream.read(&mut buffer)?;
    println!("Risposta dal server: {}", String::from_utf8_lossy(&buffer[..size]));

    Ok(())
}
```



Mail slot (Windows)

- Una mail slot in Windows è un meccanismo di IPC
- E' una coda di messaggi unidirezionale, dove il mittente scrive un messaggio che viene letto dal destinatario
- La mail slot viene creata specificando un nome
- Il mittente si identifica attraverso il metodo `MailslotClient`
- Il destinatario si identifica attraverso il metodo `MailslotServer`
- Note:
 - occorre prima creare il lato destinazione (che si mette in attesa) e poi il lato mittente
 - In `cargo.toml`
 - `[dependencies]`
 - `mail_slot = "0.1.3"`

```

use std::thread;
use std::time::Duration;
use mail_slot::{MailslotServer, MailslotName};
fn main() {
    // Crea un nome per il mailslot
    let name = MailslotName::local("naive");

    // Crea un server mailslot
    let mut server = MailslotServer::new(&name).unwrap();

    println!("Server is running, waiting for messages...");
    loop {
        thread::sleep(Duration::from_millis(2000));
        match server.get_next_unread(){
            Ok(maybe_msg) => {
                match maybe_msg {
                    Some(msg) => {
                        let text = String::from_utf8_lossy(&msg);
                        if text == "STOP" {
                            println!("Stop from client");
                            break;
                        }
                        println!("[Server] Ricevuto: {}", text);
                    }
                    None => {
                        println!("[Server] Nessun messaggio disponibile");
                        continue;
                    }
                }
            }
            Err(e) => {
                eprintln!("[Server] Errore nella lettura: {}", e);
                break;
            }
        }
    }
}

```

La funzione di lettura non è bloccante

Il protocollo richiede la ricezione di un messaggio di chiusura

mailslot_server.rs

```
use mail_slot::{MailslotName, MailslotClient};
use std::thread;
use std::time::Duration;

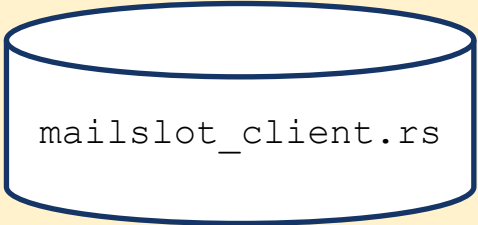
fn main() {
    // Crea un nome per il mailslot
    let name = MailslotName::local("naive");

    // Crea un client mailslot
    let mut client = MailslotClient::new(&name).unwrap();

    for i in 1..=5 {
        let message = format!("Messaggio #{}", i);
        client.send_message(message.as_bytes()).unwrap();

        println!("[Client] Inviato: {}", message);
        thread::sleep(Duration::from_millis(500)); // Simula tempo tra i messaggi
    }

    let message = format!("STOP");
    client.send_message(message.as_bytes()).unwrap();
    println!("[Client] Inviato: {}", message);
}
```



mailslot_client.rs

Named Pipe (Windows)

- E' una coda di messaggi **bidirezionale**
- La lettura è bloccante
- La pipe viene chiusa dal client uscendo dallo scope
- Quando il client chiude la pipe il server riceve un buffer di 0 byte
- Un server può creare più istanze della pipe (per più client)
- Note:
 - occorre prima attivare il server e poi il client
 - In `cargo.toml`
 - `[dependencies]`
 - `named_pipe = "0.4.1"`

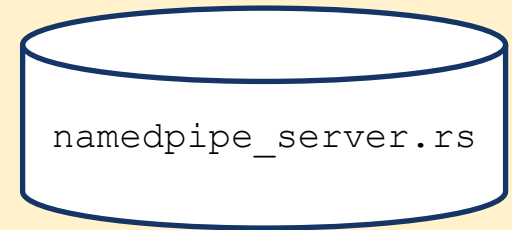
```

use named_pipe::PipeOptions;
use std::io::Read;
fn main() {
    let pipe = PipeOptions::new("\\\\.\\pipe\\test_pipe")
        .single()
        .unwrap();

    println!("In attesa di un client...");
    let mut connection = pipe.wait().unwrap();
    println!("Client connesso.");

    let mut buffer = [0u8; 128];
    loop {
        match connection.read(&mut buffer) {
            Ok(0) => {
                println!("Client ha chiuso la pipe.");
                break;
            }
            Ok(n) => {
                println!("{}", String::from_utf8_lossy(&buffer[..n]));
            }
            Err(e) => {
                eprintln!("Errore nella lettura: {}", e);
                break;
            }
        }
    }
}

```



```
use std::io::Write;
use std::fs::OpenOptions;

fn main() {
    let mut pipe = OpenOptions::new()
        .write(true)
        .open("\\\\.\\pipe\\test_pipe")
        .unwrap();

    for i in 0..10 {
        let msg = format!("Numero {}\\n", i);
        pipe.write_all(msg.as_bytes()).unwrap();
        print!("Messaggio inviato: {}", msg);
    }
}
```



Esempio di uso bidirezionale

```
use named_pipe::PipeOptions;
use std::io::{Read, Write};

fn main() {
    let pipe = PipeOptions::new("\\\\.\\pipe\\pipe_demo")
        .single()
        .unwrap();

    println!("[Server] In attesa del client...");
    let mut conn = pipe.wait().unwrap();
    println!("[Server] Client connesso.");

    let mut buffer = [0u8; 128];
    let n = conn.read(&mut buffer).unwrap();
    let ricevuto = String::from_utf8_lossy(&buffer[..n]);
    println!("[Server] Ricevuto: {}", ricevuto.trim());

    let risposta = b"Ciao dal server!\n";
    conn.write_all(risposta).unwrap();
    println!("[Server] Risposta inviata.");
}
```

namedpipe_server_bid.rs

```
use std::fs::OpenOptions;  
use std::io::{Read, Write};
```

```
fn main() {  
    println!("[Client] Connessione alla pipe...");  
    let mut pipe = OpenOptions::new()  
        .read(true)  
        .write(true)  
        .open("\\\\.\\pipe\\pipe_demo")  
        .unwrap();  
  
    let messaggio = b"Ciao dal client!\n";  
    pipe.write_all(messaggio).unwrap();  
    println!("[Client] Messaggio inviato.");  
  
    let mut buffer = [0u8; 128];  
    let n = pipe.read(&mut buffer).unwrap();  
    let risposta = String::from_utf8_lossy(&buffer[..n]);  
    println!("[Client] Risposta ricevuta: {}", risposta.trim());  
}
```

namedpipe_client_bid.rs

Scambiare messaggi strutturati tra processi in Rust

- Il crate **serde** offre le funzionalità necessarie a serializzare e deserializzare buona parte delle strutture dati Rust usano uno qualunque tra i seguenti formati esterni
 - JSON, Bincode, CBOR, YAML, MessagePack, TOML, Pickle, RON, BSON, Avro, JSON5, Postcard, URL query strings, Envy, S-expressions, D-Bus's binary wire format, FlexBuffers, Bencode, DynamoDB Items, Hjson, ...
 - Per ciascuno di questi formati esiste un apposito crate che deve essere incluso insieme a quello base
- La libreria estende la macro **`#[derive(Serialize, Deserialize)]`** per aggiungere in modo automatico il supporto alle operazioni di conversione a tipi definiti dall'utente
 - Occorre includere il crate "serde" con la relativa versione e abilitare la funzionalità "derive"
 - **`serde = {version = "1.0.137", features = ["derive"]}`**
 - **`serde_json = "1.0.140"`**

```
use serde::{Serialize, Deserialize};
#[derive(Serialize, Deserialize, Debug)]
struct Point {x: i32, y: i32}
#[derive(Serialize, Deserialize, Debug)]
struct X(i32, i32);
#[derive(Serialize, Deserialize, Debug)]
struct Y(i32);
#[derive(Serialize, Deserialize, Debug)]
struct Z;

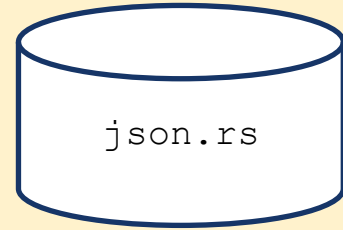
fn main() {
    let point = Point { x: 1, y: 2 };
    let x = X(0, 0);
    let y = Y(0);
    let z = Z;

    // Convert the Point to a JSON string.
    let serialized = serde_json::to_string(&point).unwrap();
    println!("serialized = {}", serialized); // Prints serialized = {"x":1,"y":2}
    // Convert the JSON string back to a Point.
    let deserialized: Point = serde_json::from_str(&serialized).unwrap();
    println!("deserialized = {:?}", deserialized); // Prints deserialized = Point { x: 1, y: 2 }

    let serialized = serde_json::to_string(&x).unwrap();
    println!("serialized = {}", serialized); // Prints serialized = [0,0]
    let deserialized: X = serde_json::from_str(&serialized).unwrap();
    println!("deserialized = {:?}", deserialized); // Prints deserialized = X(0, 0)

    let serialized = serde_json::to_string(&y).unwrap();
    println!("serialized = {}", serialized); // Prints serialized = 0
    let deserialized: Y = serde_json::from_str(&serialized).unwrap();
    println!("deserialized = {:?}", deserialized); // Prints deserialized = Y(0)

    let serialized = serde_json::to_string(&z).unwrap();
    println!("serialized = {}", serialized); // Prints serialized = null
    let deserialized: Z = serde_json::from_str(&serialized).unwrap();
    println!("deserialized = {:?}", deserialized); // Prints deserialized = Z
}
```

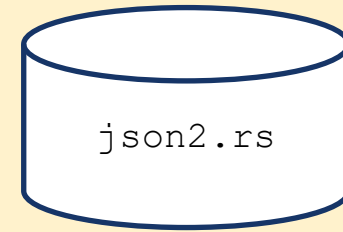


```
use serde::{Serialize, Deserialize};
```

```
#[derive(Serialize, Deserialize, Debug)]
```

```
enum E {  
    W { a: i32, b: i32 },  
    X(i32, i32),  
    Y(i32),  
    Z,  
}
```

```
fn main() {  
    let w = E::W { a: 0, b: 0 }; // Rappresentato come l'oggetto `{"W":{"a":0,"b":0}}`  
    let x = E::X(0, 0);          // Rappresentato come l'oggetto `{"X":[0,0]}`  
    let y = E::Y(0);             // Rappresentato come l'oggetto `{"Y":0}`  
    let z = E::Z;                // Rappresentato come la stringa `"Z"`  
  
    let serialized = serde_json::to_string(&w).unwrap();  
    println!("serialized = {}", serialized); // Prints serialized = {"W":{"a":0,"b":0}}  
    let deserialized: E = serde_json::from_str(&serialized).unwrap();  
    println!("deserialized = {:?}", deserialized); // Prints deserialized = W { a: 0, b: 0 }  
  
    let serialized = serde_json::to_string(&x).unwrap();  
    println!("serialized = {}", serialized); // Prints serialized = {"X":[0,0]}  
    let deserialized: E = serde_json::from_str(&serialized).unwrap();  
    println!("deserialized = {:?}", deserialized); // Prints deserialized = X(0, 0)  
  
    let serialized = serde_json::to_string(&y).unwrap();  
    println!("serialized = {}", serialized); // Prints serialized = {"Y":0}  
    let deserialized: E = serde_json::from_str(&serialized).unwrap();  
    println!("deserialized = {:?}", deserialized); // Prints deserialized = Y(0)  
  
    let serialized = serde_json::to_string(&z).unwrap();  
    println!("serialized = {}", serialized); // Prints serialized = "Z"  
    let deserialized: E = serde_json::from_str(&serialized).unwrap();  
    println!("deserialized = {:?}", deserialized); // Prints deserialized = Z  
}
```



Serde + Mail Slot

- Prendendo spunto dai programmi client/server che comunicano tramite Mail Slot si riportano le versioni che fanno marshalling e unmarshalling per mezzo di serde.

```

use mail_slot::{MailslotServer, MailslotName};
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug)]
struct Point {x: i32, y: i32}

fn main() {
    let name = MailslotName::local("naive");

    // Crea un server mailslot
    let mut server = MailslotServer::new(&name).unwrap();

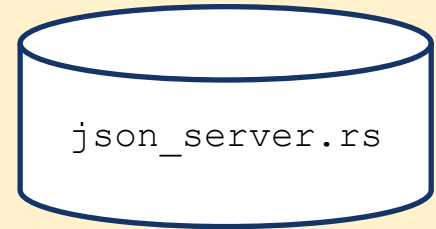
    println!("Server is running, waiting for messages...");

    'outer: loop {
        while let Some(msg) = server.get_next_unread().unwrap() {
            let msg_str = String::from_utf8(msg).unwrap();

            if msg_str == "STOP" {
                println!("Stop from client");
                break 'outer;
            }

            let deserialized: Point= serde_json::from_str(&msg_str).unwrap();
            println!("message from client {}", msg_str);
            println!("deserialized = {:?}", deserialized);
        }
    }
}

```



```

use serde::{Serialize, Deserialize};
#[derive(Serialize, Deserialize, Debug)]
struct Point {x: i32, y: i32}

use mail_slot::{MailslotName, MailslotClient};

fn main() {
    let name = MailslotName::local("naive");

    let mut client = MailslotClient::new(&name).unwrap();

    let point1 = Point { x: 1, y: 2 };
    let point2 = Point { x: 2, y: 1 };

    // Convert the Point to a JSON string.
    let serialized = serde_json::to_string(&point1).unwrap();
    println!("serialized = {}", serialized); // Prints serialized = {"x":1,"y":2}

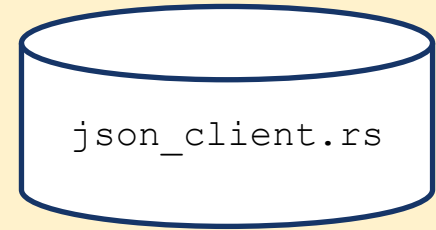
    // Write the message to the mail slot
    client.send_message(serialized.as_bytes()).unwrap();

    let serialized = serde_json::to_string(&point2).unwrap();
    println!("serialized = {}", serialized); // Prints serialized = {"x":2,"y":1}

    // Write the message to the mail slot
    client.send_message(serialized.as_bytes()).unwrap();

    client.send_message(b"STOP").unwrap();
}

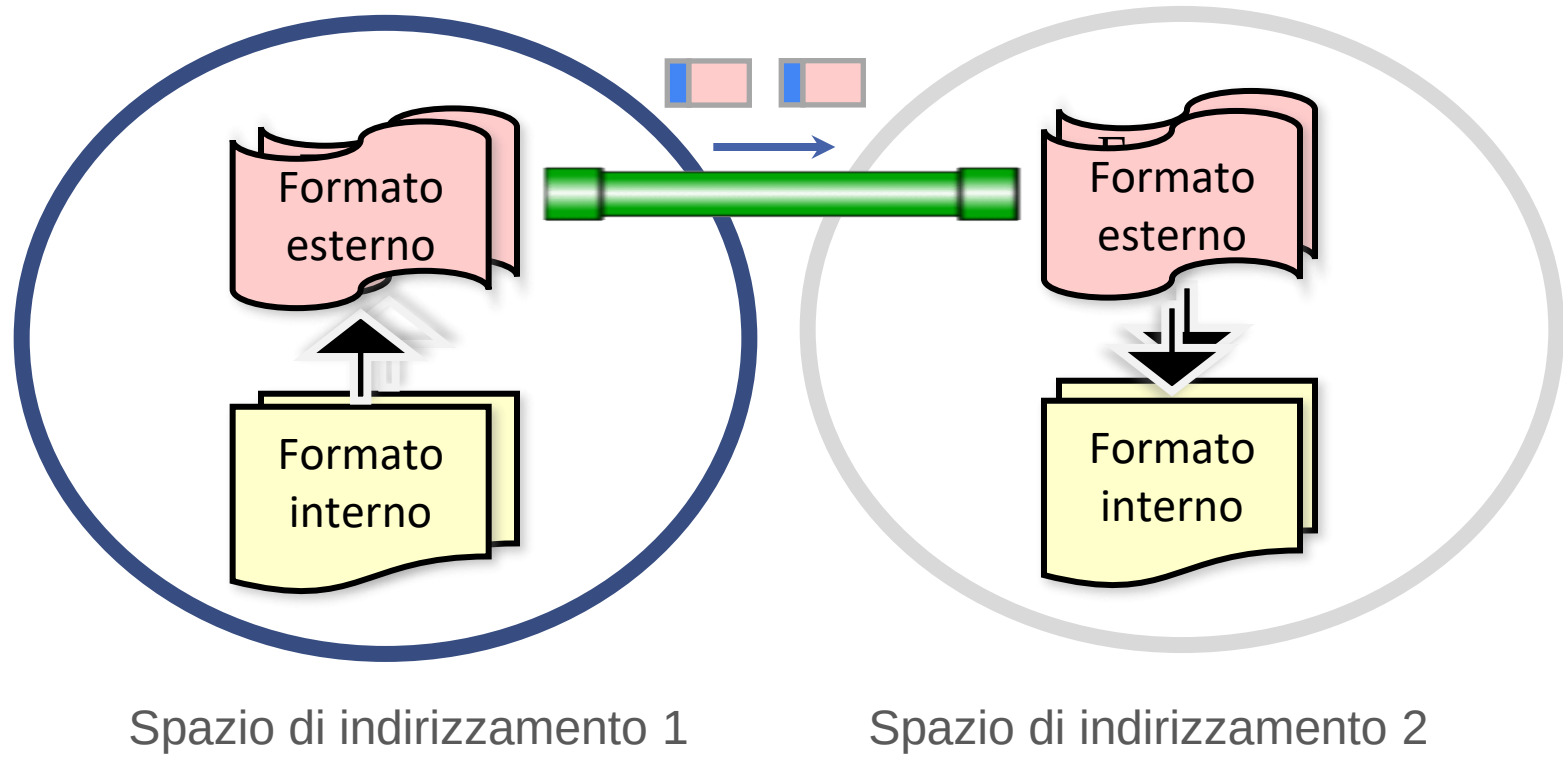
```



Pipe

- Possono essere utilizzate quando c'è una relazione padre/figlio tra processi
- «Tubi» che permettono il trasferimento di sequenze di byte di dimensioni arbitrarie
 - Occorre inserire marcatori che consentano di delimitare i singoli messaggi
 - Comunicazione sincrona 1-1
- Implementati come buffer all'interno della memoria del kernel
 - Disponibili sia in Linux che Windows con alcune differenze sul modo di fornire i descrittori ai processi derivati
 - Il contenuto della pipe non è segmentato a blocchi

Pipe



Pipe

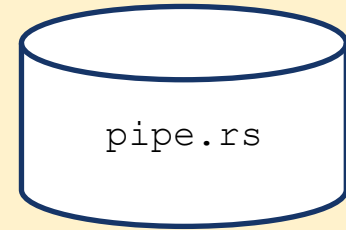
- La pipe è vista come un file descriptor (in scrittura per l'estremità entrante e in lettura per l'estremità uscente)
- In Unix: il file descriptor è ereditato grazie alla `fork()`, i singoli processi chiudono l'estremità che non usano
- In C/ C++, una pipe può essere creata da programma con le funzioni
 - `BOOL CreatePipe(
HANDLE *pHRead,
HANDLE *pHWrite,
SECURITY_ATTRIBUTES *lpPipeAttributes,
DWORD nSize)`
 - Windows (`#include <namedpipeapi.h>`)
 - `int pipe(int fd[2])`
 - Linux (`#include <unistd.h>`)
- Si legge e scrive da una pipe con le operazioni di lettura/scrittura del sistema operativo
 - `FileRead(...)` / `FileWrite(...)` – Windows
 - `read(...)` / `write(...)` – Linux
- Si chiude una pipe con la funzione corrispondente
 - `CloseHandle(...)` / `close(...)`

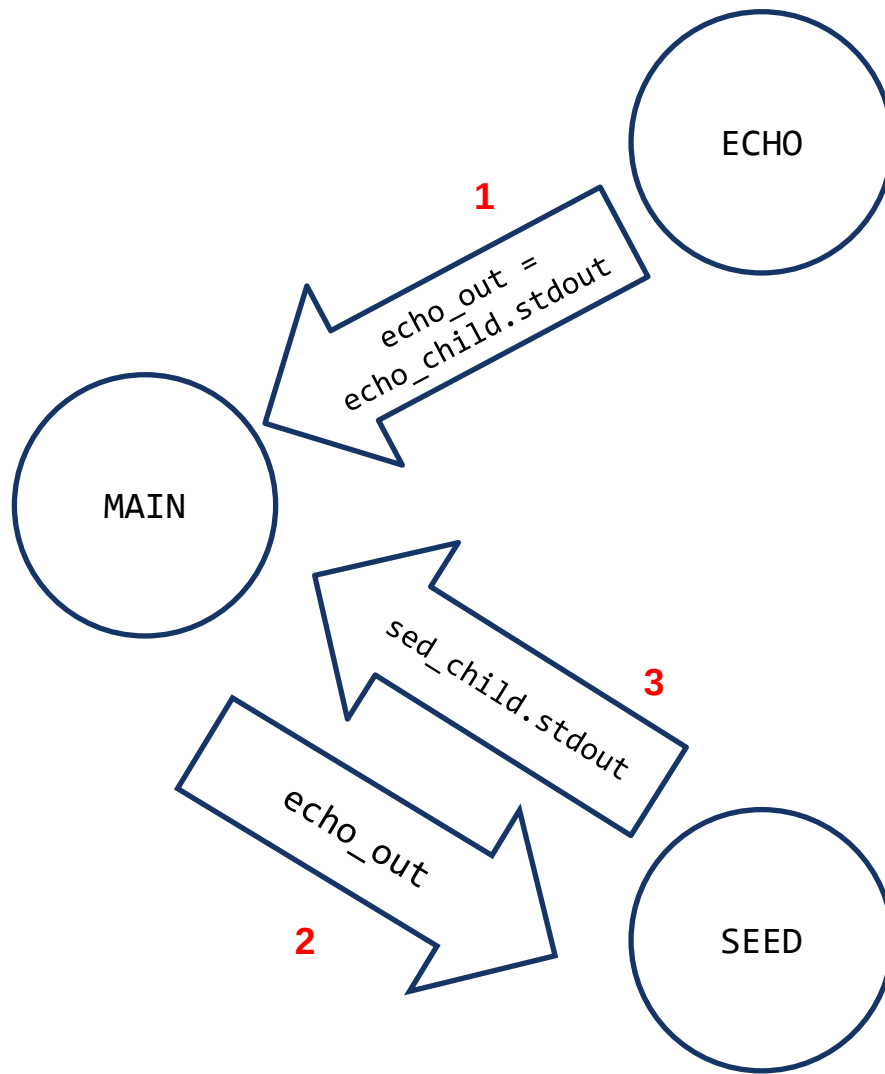
```
use std::process::Stdio;
use std::process::Command;
fn main(){
    let echo_child = Command::new("echo")
        .arg("Ciao Mama")
        .stdout(Stdio::piped())
        .spawn()
        .expect("Failed to start echo process");

    let echo_out = echo_child.stdout.expect("Failed to open echo stdout");

    let sed_child = Command::new("sed")
        .arg("s/Mama/Mamma/")
        .stdin(Stdio::from(echo_out))
        .stdout(Stdio::piped())
        .spawn()
        .expect("Failed to start sed process");

    let output = sed_child.wait_with_output().expect("Failed to wait on sed");
    println!("{}", String::from_utf8_lossy(&output.stdout));
}
```





```
use std::process::Command;
use std::io::prelude::*;
use std::process::Stdio;
static PANGRAM: &'static str = "Stringa di prova organizzata su 2 righe\nQuesta e' la seconda riga\n";
fn main() {
    // Spawn the wc command
    // wc counts the number of lines, words and bytes
    let process = match Command::new("wc")
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn() {
        Err(why) => panic!("couldn't spawn wc: {}", why),
        Ok(process) => process,
    };

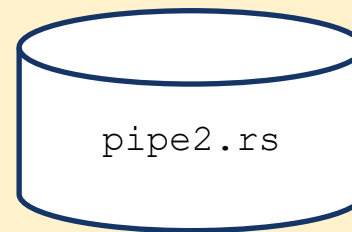
    // Write a string to the stdin of wc.
    match process.stdin.unwrap().write_all(PANGRAM.as_bytes()) {
        Err(why) => panic!("couldn't write to wc stdin: {}", why),
        Ok(_) => println!("sent my string to wc"),
    }

    // Because stdin does not live after the above calls, it is dropped, and the pipe is closed.
    // This is very important, otherwise wc wouldn't start processing the input we just sent.

    let mut s = String::new();
    match process.stdout.unwrap().read_to_string(&mut s) {
        Err(why) => panic!("couldn't read wc stdout: {}", why),
        Ok(_) => print!("wc responded with:\n{}", s),
    }
}
```



```
use std::fs::File;
use std::io::{self, Read, Write};
use std::process::{Command, Stdio};
use std::thread;
fn main() -> io::Result<()> {
    let filename = "test.txt";
    // Creare un thread per leggere il contenuto del file
    let handle = thread::spawn(move || -> io::Result<String> {
        let mut file = File::open(filename)?;
        let mut contents = String::new();
        file.read_to_string(&mut contents)?;
        Ok(contents)
    });
    let mut child = Command::new("rev") // Creare un processo per eseguire il comando rev
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn()
        .expect("Failed to spawn child process");
    // Attendere che il thread finisca e ottenere il contenuto del file
    let file_contents = handle.join().unwrap()?;
    // Ottenere lo stdin del processo e scrivere il contenuto del file
    if let Some(mut stdin) = child.stdin.take() {
        stdin.write_all(file_contents.as_bytes())?;
    }
    // Ottenere lo stdout del processo e leggere l'output
    let output = child.wait_with_output()?;
    println!("Output:\n{}", String::from_utf8_lossy(&output.stdout));
    Ok(())
}
```



```
use std::io::{self, Write};
use std::process::{Command, Stdio};
use std::thread;
use std::time::Duration;
```

Pipe: comunicazione semisincrona

```
const BLOCCO:usize = 8192;
fn main() -> io::Result<()> {
    println!("Avvio del programma: Test di blocco della scrittura su pipe.");

    // Creiamo un processo figlio che gestirà la lettura dalla pipe.
    let mut child = Command::new("reader_pipe")
        .env("PATH", "./target/debug/")
        // Lo stdout del figlio verrà rediretto alla pipe di scrittura.
        .stdin(Stdio::piped()) // Reindirizza lo stdin del figlio alla pipe (lato scrittura del padre)
        // .stdout(Stdio::null()) // Se non ci interessa l'output del figlio in questo test
        .spawn()?;

    // Otteniamo il writer per lo stdin del figlio (che è il lato scrittura della pipe per il padre)
    let mut child_stdin = child.stdin.take().expect("Impossibile ottenere stdin dal figlio");

    println!("Processo figlio avviato. Inizio scrittura da parte del padre.");
    println!("Il padre scriverà continuamente nella pipe. Il blocco indicherà che la pipe è piena.");
```

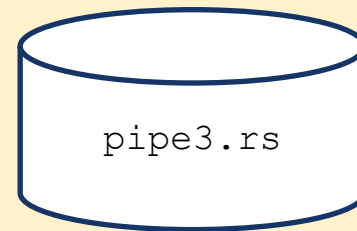


segue


```

let mut bytes_written: usize = 0;
let mut count = 0;
loop {
    match child_stdin.write_all(b"A") {
        Ok(_) => {
            bytes_written += 1;
            if bytes_written % BLOCCO == 0 {
                print!("."); // Stampa un puntino ogni BLOCCO KB scritto
                io::stdout().flush()?;
                count += 1;
            }
            thread::sleep(Duration::from_nanos(1));
        }
        Err(e) => {
            eprintln!("\nErrore durante la scrittura: {}", e);
            break;
        }
    }
    // Per osservare il blocco, non facciamo leggere il figlio per un po' di tempo.
    if count == 10 {
        break;
    }
}
println!("\nScrittura terminata. Byte scritti: {}", bytes_written);
// Per terminare il figlio in modo pulito e non lasciare processi orfani
drop(child_stdin); // Chiude il lato scrittura della pipe per il padre, segnalando EOF al figlio
child.wait()?; // Aspetta che il figlio termini
println!("Programma terminato.");
Ok(())
}

```

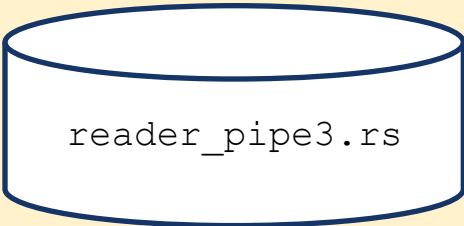


```
use std::time::Duration;
use std::thread;
use std::io::{self, Read, Write};
```

```
fn main() -> io::Result<()> {
    let mut buffer = [0; 1024]; // Buffer di 1024 bytes
    let stdin = io::stdin();
    let mut handle = stdin.lock(); //stdin.lock() per ottenere un handle esclusivo allo stdin

    let mut stdout = io::stdout();

    println!("Attendo 60 secondi prima di iniziare...");
    thread::sleep(Duration::from_secs(60));
    println!("Riparto ...");
    loop {
        match handle.read(&mut buffer) {
            Ok(0) => break, // EOF (fine dell'input)
            Ok(n) => {
                stdout.write_all(&buffer[..n])?;
                stdout.flush()?;
            }
            Err(e) => {
                eprintln!("Errore di lettura: {}", e);
                break;
            }
        }
    }
    println!("Output: Fine");
    Ok(())
}
```



reader_pipe3.rs

client/server + Pipe

- Il client manda un numero al server per confermare la sua presenza
- Il server risponde con un ACK per avviare l'elaborazione
- Il client manda al server un numero casuale tra 0 e 9
- Il server deve indovinare il numero:
 - se indovina il numero manda al client il messaggio STOP e il client termina,
 - altrimenti manda al client il messaggio CONTINUE e il client fornisce un nuovo numero casuale tra 0 e 9
- Il server attende che il client termini il processo.

```
use std::io::{Read, Write};
use std::process::{Command, Stdio};
use rand::Rng;
fn main() {
    let mut child = Command::new("client") // Avvia il processo figlio (client)
        .env("PATH", "./target/debug/")
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn()
        .expect("Failed to start child process");

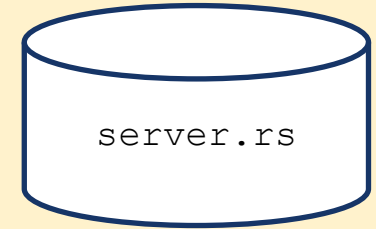
    let mut rng = rand::rng();

    let mut pipe_reader = child.stdout.take().expect("Failed to open stdout");
    let mut pipe_writer = child.stdin.take().unwrap();

    let mut number_str = [0; 2]; // ricevo un numero a singola cifra e il carattere newline
    pipe_reader.read_exact(&mut number_str).expect("Failed to read number");

    println!("Il client è vivo!");

    // Continua
```

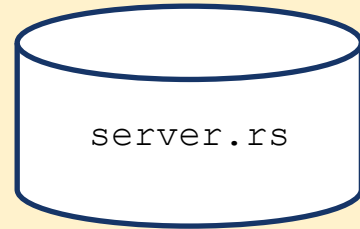


```

loop {
    // Invia un acknowledge al client
    pipe_writer.write_all("ACK\n".as_bytes()).expect("Failed to send acknowledge");

    let mut number_str = [0; 2];
    pipe_reader.read_exact(&mut number_str).expect("Failed to read number");
    let number = std::str::from_utf8(&number_str).unwrap().trim();
    println!("Ricevuto dal client:{}", number);
    let random_number = rng.random_range(0..=9);
    println!("Mio numero: {}", random_number);
    if random_number == number.parse::<i32>().unwrap(){
        pipe_writer.write_all("STOP\n".as_bytes()).expect("Failed to send acknowledge");
        break;
    }
}
// Attendere che il processo figlio termini
child.wait().expect("Failed to wait for child process");
}

```



```
use std::io;
use rand::Rng;

fn main() {

    let mut rng = rand::thread_rng();

    let random_number = rng.gen_range(0..=9);

    // Scrive il numero casuale nello stdout
    println!("{}", random_number);

    loop {
        let mut response = String::new();
        io::stdin().read_line(&mut response).expect("Failed to read response");

        if response != "ACK\n" {
            break;
        }

        let random_number = rng.gen_range(0..=9);

        // Scrive il numero casuale nello stdout
        println!("{}", random_number);
    }
}
```



Client/Server + Serde + Pipe

- Prendendo spunto dai programmi client/server che comunicano tramite pipe si riportano le versioni che fanno marshalling e unmarshalling per mezzo di serde.

```

use serde::{Serialize, Deserialize};
use std::io::Read;
use std::process::{Command, Stdio};

#[derive(Serialize, Deserialize, Debug)]
struct Point {x: i32, y: i32}

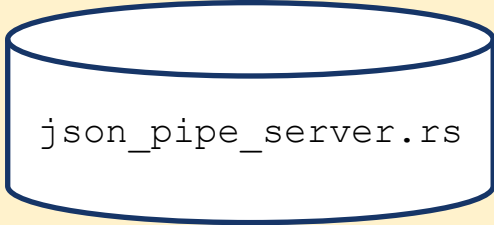
fn main() {
    let mut child = Command::new("client") // Avvia il processo figlio (client)
        .env("PATH", "./target/debug/")
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn()
        .expect("Failed to start child process");

    let mut pipe_reader = child.stdout.take().expect("Failed to open stdout");

    let mut s = String::new();
    match pipe_reader.read_to_string(&mut s) {
        Err(why) => panic!("couldn't read client stdout: {}", why),
        Ok(_) => print!("Client responded with:\n{}", s),
    }
    let deserialized: Point= serde_json::from_str(&s).unwrap();
    println!("message from client {}", s);
    println!("deserialized = {:?}", deserialized);

    // Attendere che il processo figlio termini
    child.wait().expect("Failed to wait for child process");
}

```



json_pipe_server.rs

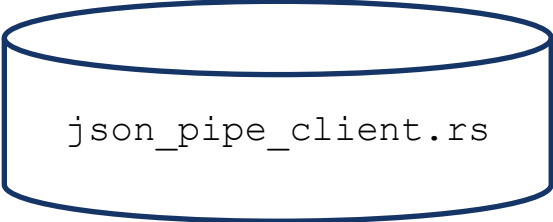

```
use serde::{Serialize, Deserialize};
#[derive(Serialize, Deserialize, Debug)]
struct Point {x: i32, y: i32}

fn main() {

    let point1 = Point { x: 1, y: 2 };

    // Convert the Point to a JSON string.
    let serialized = serde_json::to_string(&point1).unwrap();
    println!("{}", serialized); // Prints serialized = {"x":1,"y":2}

}
```



json_pipe_client.rs

Modulo std::net

- Il modulo std::net offre le primitive necessarie per lavorare con i socket di rete in modo sincrono: le operazioni di rete (come la connessione, l'invio o la ricezione dei dati) bloccano il thread corrente fino al completamento dell'operazione.
- Le strutture principali includono:
 - TcpStream: rappresenta una connessione TCP
 - TcpListener: utilizzato per avviare un server TCP che ascolta le connessioni in entrata. Accetta nuove connessioni e restituisce un TcpStream per ciascuna
 - UdpSocket: permette di inviare e ricevere pacchetti UDP
 - IpAddr, Ipv4Addr, Ipv6Addr: tipi per rappresentare indirizzi IP
 - SocketAddr, SocketAddr, SocketAddrV6: tipi per rappresentare indirizzi di socket (combinazione di indirizzo IP e porta).

Tcpstream

```
use std::io::{Read, Write};
use std::net::TcpStream;

fn main() {
    let mut stream = TcpStream::connect("127.0.0.1:8080").expect("Errore nella connessione al server");

    let num: i32 = 42; // Numero da inviare al server
    stream.write_all(&num.to_be_bytes()).expect("Errore nell'invio del numero");

    let mut buffer = [0; 4];
    stream.read_exact(&mut buffer).expect("Errore nella lettura della risposta");

    let result = i32::from_be_bytes(buffer);
    println!("Risposta dal server: {}", result);
}
```



```

use std::io::{Read, Write};
use std::net::{TcpListener, TcpStream};

fn handle_client(mut stream: TcpStream) {
    let mut buffer = [0; 4];
    stream.read_exact(&mut buffer).expect("Errore nella lettura del buffer");

    let num = i32::from_be_bytes(buffer);

    println!("Ricevuto {}", num);
    let doubled = num * 2;

    stream.write_all(&doubled.to_be_bytes()).expect("Errore nell'invio della risposta");
}

fn main() {
    let listener = TcpListener::bind("127.0.0.1:8080").expect("Errore nel binding del listener");
    println!("Server in ascolto su 127.0.0.1:8080...");
    for stream in listener.incoming() { // rimane sempre in ascolto per accettare nuovi client
        match stream {
            Ok(stream) => {
                std::thread::spawn(|| {
                    handle_client(stream);
                });
            }
            Err(e) => {
                eprintln!("Errore nell'accettare la connessione: {}", e);
            }
        }
    }
}

```



tcpserver.rs

```

use std::io::{Read, Write};
use std::net::{TcpListener, TcpStream};
fn handle_client(mut stream: TcpStream) {
    let mut buffer = [0; 4];
    stream.read_exact(&mut buffer).expect("Errore nella lettura del buffer");

    let num = i32::from_be_bytes(buffer);

    println!("Ricevuto {}", num);
    let doubled = num * 2;

    stream.write_all(&doubled.to_be_bytes()).expect("Errore nell'invio della risposta");
}
fn main() {
    let listener = TcpListener::bind("127.0.0.1:8080").expect("Errore nel binding del listener");
    println!("Server in ascolto su 127.0.0.1:8080...");
    for stream in listener.incoming() { // rimane sempre in ascolto per accettare nuovi client
        match stream {
            Ok(stream) => {
                std::thread::spawn(|| {
                    handle_client(stream);
                }).join().unwrap();
                break; // versione per chiudere la connessione al primo client
            }
            Err(e) => {
                eprintln!("Errore nell'accettare la connessione: {}", e);
            }
        }
    }
}

```



Marshalling/Unmarshalling con Protobuf

- Protocol Buffers (abbreviato in Protobuf) è un metodo efficiente ed automatico per serializzare dati strutturati (sviluppato da Google....)
- Modo per definire come i dati sono strutturati e poi generare automaticamente il codice per creare, leggere e scrivere quei dati in un **formato binario** compatto e veloce
- Protobuf fornisce un linguaggio di definizione dati neutrale e fornisce strumenti per generare il codice specifico **per ogni linguaggio**
- Componenti chiave:
 - File di definizione della struttura dati (.proto)
 - File di testo che descrive la struttura dati usando la sintassi di Protobuf
 - Compilatore Protobuf (protoc)
 - Prende il file .proto come input e genera il codice sorgente che verrà incluso nel main
 - Programma di Serializzazione/Deserializzazione

Definire la struttura dati

- Si utilizza un linguaggio neutro definito da Protobuf.
- I file .proto son posizionati in una cartella proto sotto src oppure in src/

```
// src/user.proto
syntax = "proto3";

package myapp;

message MyData {
  string message = 1;
  int32 value = 2;
  bool is_active = 3;
}
```



- syntax specifica la versione di Protobuf
- package definisce il modulo per il codice generato da Protobuf (myapp.rs)
- message definisce la struttura dati

Configurare il Progetto Rust in Cargo.toml

- Occorre inserire in Cargo.toml le librerie da usare per lavorare con Protobuf, aggiungendo le dipendenze necessarie

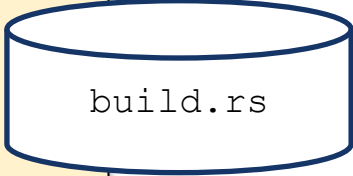
```
[dependencies]
prost = "0.12"          # Libreria runtime per
serializzazione/deserializzazione
prost-types = "0.12"
```

```
[build-dependencies]
prost-build = "0.12"    # Libreria per la generazione del codice
durante la build
```


Creare lo Script di Build

- Questo script speciale viene eseguito da Cargo prima della compilazione del codice Rust principale.
- Il suo compito è generare il codice Rust a partire dalle definizioni indicate nel file .proto.
- Il file build.rs va salvato nella root del progetto (accanto a Cargo.toml)
- Il codice generato verrà depositato in una directory temporanea gestita da Cargo (es. target/debug/build/.../out/myapp.rs).

```
fn main() -> Result<(), Box<dyn std::error::Error>> {  
    // Indica a prost-build quali file .proto compilare e dove cercare le  
    importazioni.  
    // I file .proto si trovano in "src/", quindi forniamo "src/" come radice.  
    prost_build::compile_protos(  
        &["src/user.proto"], // Il file .proto  
        &["src/"]           // La directory radice per la ricerca dei file .proto  
    )?;  
    Ok(())  
}
```



build.rs

Installare il Compilatore Protobuf (protoc)

- prost-build invoca il compilatore protoc di Google.
- Installa protoc:
 - Ubuntu: `sudo apt install protobuf-compiler`
 - Windows: da <https://github.com/protocolbuffers/protobuf/releases>.

Programma per Serializzare/Deserializzare

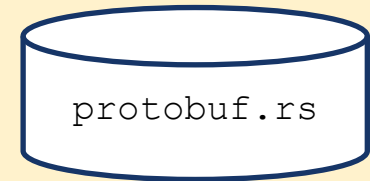
```
// Importa il modulo Rust generato dal tuo file .proto.
// Il nome del modulo (es. myapp) deriva dal package nel tuo .proto.
// include! legge il file Rust generato dalla directory OUT_DIR di Cargo.
#[allow(clippy::derive_partial_eq_without_eq)] // Sopprime l'avviso di clippy
pub mod myapp {
    // Il nome del file generato è il nome del package + ".rs"
    include!(concat!(env!("OUT_DIR"), "/myapp.rs"));
}

use prost::Message; // Importa la trait Message per i metodi encode e decode
use std::io::Write; // Per scrivere i byte serializzati in un file
use std::fs::File; // Per la gestione dei file
use std::io::Read; // Importa la trait Read per il metodo read_to_end

fn main() {
    // Creare un'istanza della tua struttura dati generata
    let my_struct_instance = myapp::MyData {
        message: "Hello Protobuf!".to_string(),
        value: 12345,
        is_active: true,
    };
    println!("Struttura originale: {:?}", my_struct_instance);
}
```

segue

59

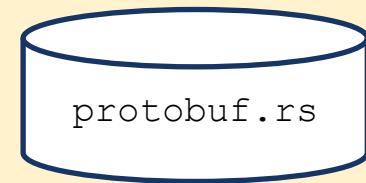


```
// encode() è un metodo fornito dalla trait prost::Message
// implementata automaticamente per le struct generate.
let mut buffer_bytes = Vec::new(); // Un vettore vuoto per contenere i byte serializzati
my_struct_instance.encode(&mut buffer_bytes)
    .expect("Errore durante la serializzazione della struttura");

println!("Bytes serializzati (formato binario Protobuf): {:?}", buffer_bytes);
println!("Dimensione dei bytes serializzati: {} byte", buffer_bytes.len());

// Salvare i byte serializzati in un file esterno
let output_filename = "my_data.bin";
let mut file = File::create(output_filename)
    .expect("Impossibile creare il file per la scrittura");
file.write_all(&buffer_bytes)
    .expect("Impossibile scrivere i byte nel file");
println!("Dati serializzati salvati in: {}", output_filename);

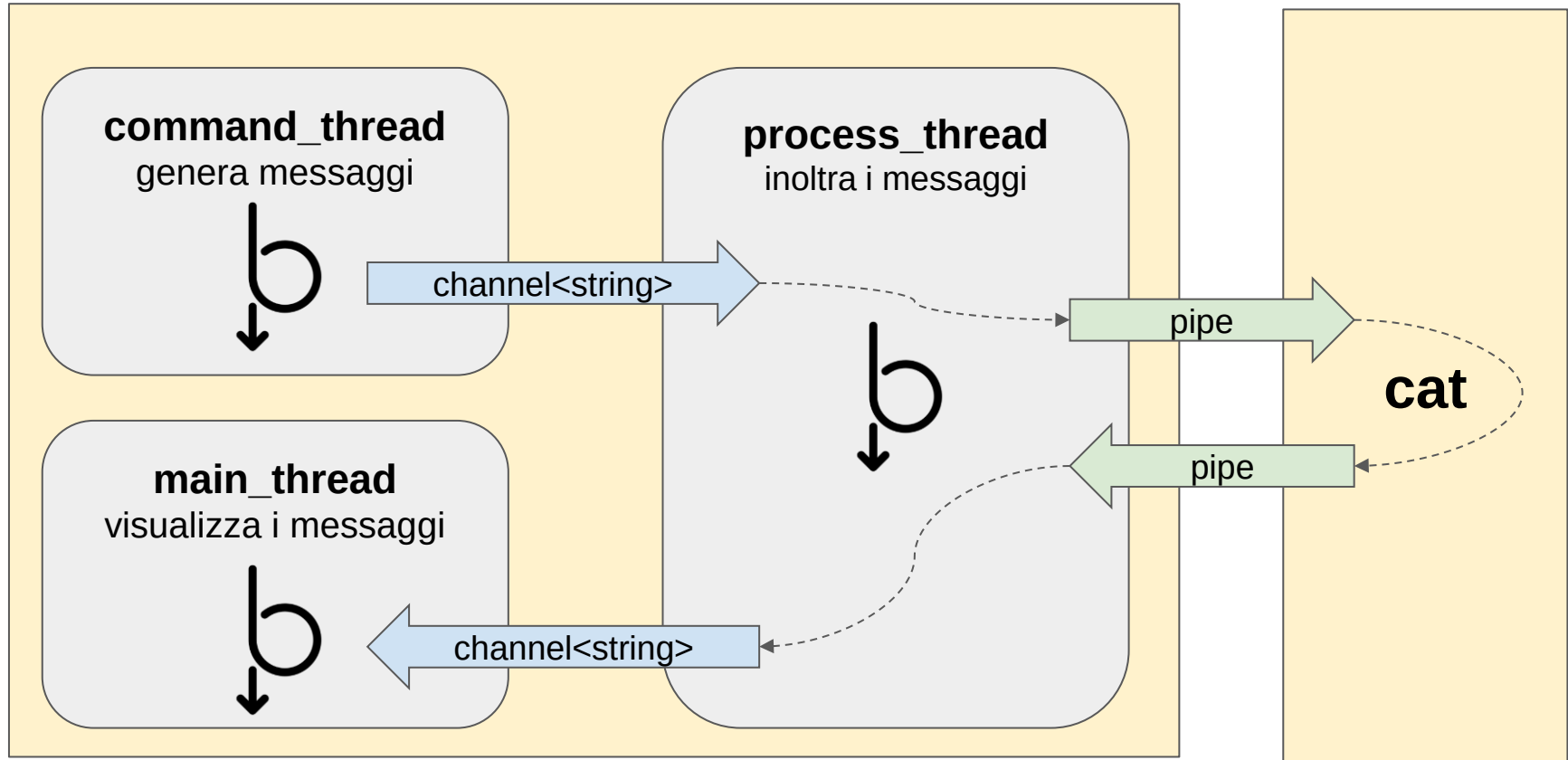
// Deserializzare i byte dal file
let mut read_file = File::open(output_filename)
    .expect("Impossibile aprire il file per la lettura");
let mut read_buffer = Vec::new();
read_file.read_to_end(&mut read_buffer)
    .expect("Impossibile leggere il file");
let deserialized_instance = myapp::MyData::decode(&read_buffer[..])
    .expect("Errore durante la deserializzazione dei dati");
println!("Struttura deserializzata: {:?}", deserialized_instance);
// Verificare che i dati siano gli stessi
assert_eq!(my_struct_instance, deserialized_instance);
}
```



Output

```
Struttura originale: MyData { message: "Hello Protobuf!", value: 12345, is_active: true }  
Bytes serializzati (formato binario Protobuf): [10, 15, 72, 101, 108, 108, 111, 32, 80, 114, 111,  
116, 111, 98, 117, 102, 33, 16, 185, 96, 24, 1]  
Dimensione dei bytes serializzati: 22 byte  
Dati serializzati salvati in: my_data.bin  
Struttura deserializzata: MyData { message: "Hello Protobuf!", value: 12345, is_active: true }
```

Comunicazione tra processi



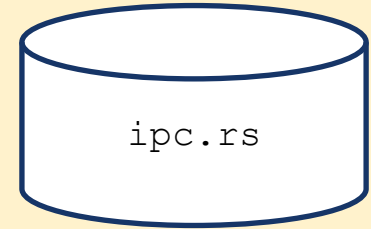
Esempio comunicazione processi

```
use std::io::{BufRead, BufReader, Write};
use std::process::{Command, Stdio};
use std::sync::mpsc::{channel, Receiver, Sender};
use std::thread;
use std::thread::sleep;
use std::time::Duration;
```

```
fn start_process(sender: Sender<String>, receiver: Receiver<String>) {

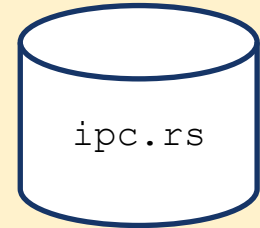
    let child = Command::new("cat")
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn()
        .expect("Failed to start process");

    //continua
```



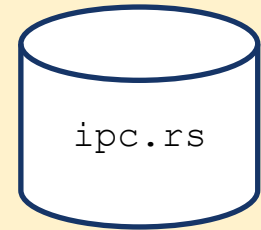
Esempio comunicazione processi

```
thread::spawn(move || {
    let mut pipe_in = BufReader::new(child.stdout.unwrap());
    let mut pipe_out = child.stdin.unwrap();
    for line in receiver {
        pipe_out.write_all(line.as_bytes()).unwrap();
        let mut buf = String::new();
        match pipe_in.read_line(&mut buf) {
            Ok(_) => {
                // inoltra quanto ricevuto dalla pipe sul canale di uscita
                sender.send(buf).unwrap();
                continue;
            }
            Err(e) => {
                println!("an error!: {:?} ", e);
                break;
            }
        }
    }
});
}
```

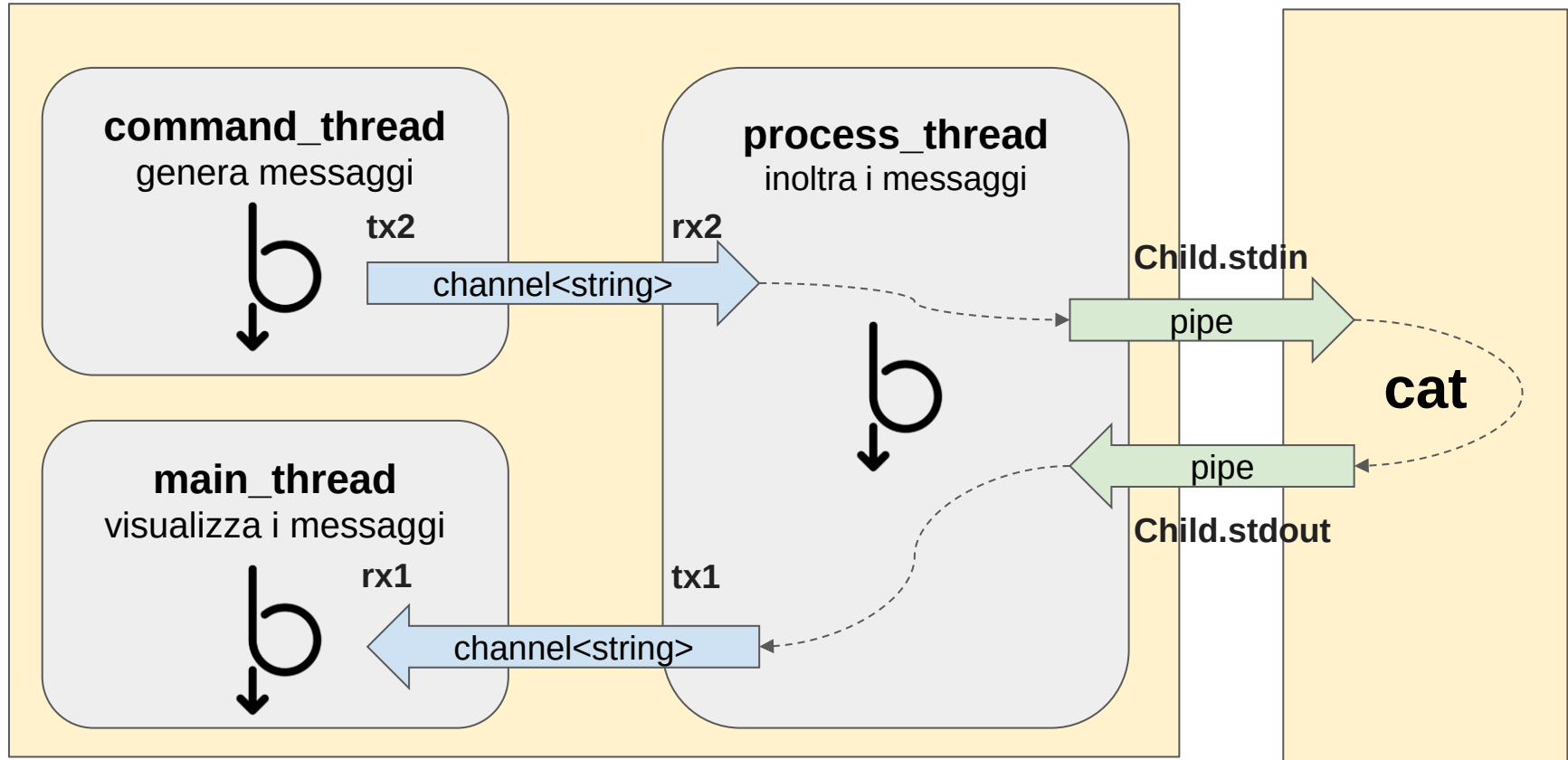


Esempio comunicazione processi

```
fn start_command_thread(sender: Sender<String>) {  
    thread::spawn(move || {  
        for i in 1..10 {  
            sleep(Duration::from_secs(3));  
            sender.send(String::from(format!("Message {} from command thread\n", i)))  
                .unwrap();  
        }  
    });  
}  
  
fn main() {  
    let (tx1, rx1) = channel();  
    let (tx2, rx2) = channel();  
  
    start_process(tx1, rx2);  
  
    start_command_thread(tx2);  
  
    rx1.iter().for_each(|line| println!("Echo process response: {}", line))  
}
```



Comunicazione tra processi



Comunicazione tra processi

- Il crate **interprocess** offre la possibilità di gestire la comunicazione tra processi tramite un'interfaccia univoca multiplatforma continuando a garantire le funzionalità specifiche dei S.O.
 - Unnamed/named pipes, Posix/C signals, socket

```
use interprocess::local_socket::{LocalSocketListener, LocalSocketStream};
use std::io::{prelude::*, BufReader};
use std::{thread, time::Duration};

const SOCKET_NAME: &str = "my-socket";

fn main() {
    // Avvia il server in un thread separato
    let server_thread = thread::spawn(move || {
        run_server().unwrap();
    });

    // Al server il tempo di partire
    thread::sleep(Duration::from_millis(100));

    // Esegue il client nel thread principale
    run_client().unwrap();

    // Aspetta la fine del server
    server_thread.join().unwrap();
}
```



```
fn run_server() -> std::io::Result<()> {
    let listener = LocalSocketListener::bind(SOCKET_NAME)?;
    println!("Server in ascolto su '{}'", SOCKET_NAME);

    let mut conn = listener.accept()?;
    println!("Server: connessione accettata");

    let mut reader = BufReader::new(&mut conn);
    let mut message = String::new();
    reader.read_line(&mut message)?;
    println!("Server: ricevuto: {}", message.trim());

    conn.write_all(b"Ciao client!\n")?;
    println!("Server: risposta inviata");
    Ok(())
}

fn run_client() -> std::io::Result<()> {
    let mut conn = LocalSocketStream::connect(SOCKET_NAME)?;
    println!("Client: connesso al server");

    conn.write_all(b"Ciao server!\n")?;
    conn.flush()?;

    let mut reader = BufReader::new(&mut conn);
    let mut response = String::new();
    reader.read_line(&mut response)?;
    println!("Client: risposta dal server: {}", response.trim());
    Ok(())
}
```



interprocess.rs

Glossario

- Rappresentazione interna: rappresentazione della struttura dato coerente con il linguaggio di programmazione adottato dal processo
- Rappresentazione esterna: rappresentazione dei dati in un linguaggio neutro interpretabile da un altro processo

Glossario

- **Marshalling:** processo di trasformazione di un oggetto o una struttura dati in un formato adatto per la trasmissione attraverso una rete. Il marshalling prende la rappresentazione interna di un oggetto e la converte in una sequenza di byte (o un formato testuale) che può essere facilmente trasferito da un processo ad un altro
- **Serializzazione:** il medesimo processo finalizzato al salvataggio su file
- **Unmarshalling/Deserializzazione:** processo inverso di trasformazione da una rappresentazione esterna ad una interna.